Applied Programming

Week 9: Tree and Graph
Data Structures and Traversal Algorithms

Finding paths from permitted connections

School of Mechanical Engineering, Soongsil University

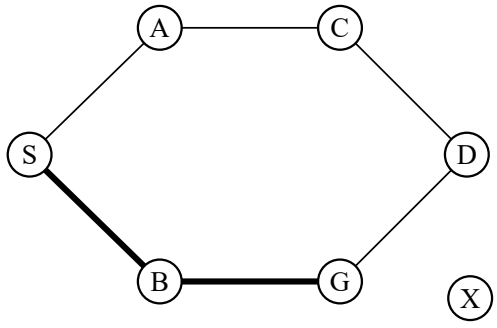
An ordered path tells a program which waypoint comes next. A collection of permitted connections does not. This chapter develops the missing operation: starting at one state, explore the available connections, record how each state was reached, and recover a path to a goal. The same node IDs will then refer to supplied robot-arm postures.

Earlier path calculations used an array whose rows were already in movement order. A path-length function measured that path, and a shortening operation removed a waypoint when a permitted connection remained. Here the order itself must be found. The input is a graph, not a plot of measured values and not an already ordered waypoint array. Lists, dictionaries, sets and deques retain their previous Python meanings. Their new role is to represent connections and manage exploration.

1 Connections, paths and trees

A **graph** consists of nodes and edges. A **node**, also called a vertex, identifies one object or state. An **edge** records a permitted connection between two nodes. The labels `S`, `A` and `G` are identifiers. Their letters do not encode coordinates, distances or joint angles. A **path** is an ordered sequence of nodes whose consecutive pairs are connected in the required direction.

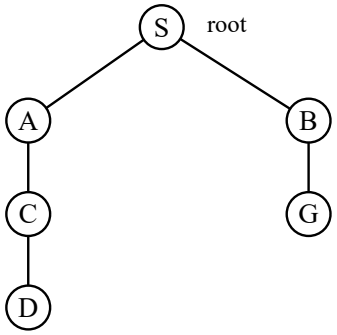
Graph: alternative connections



`Path S - B - G: 2 edges`

X has no connection to the other nodes.

Rooted tree: one parent each



`parent[G] = B`

Same connected component, without D - G.

Figure 1: Left: bold edges form the two-edge path S–B–G. Other connections remain available, and X is isolated. Right: removing D–G from the six-node connected component gives one rooted tree. Positions and line lengths are not physical measurements.

The notation $G = (V, E)$ names a graph by its node set V and edge set E. Here G in the formula means the whole graph, while `"G"` in code is the goal node's ID. The counts $|V|$ and $|E|$ mean the number of nodes and edges. An edge joins adjacent nodes, also called neighbors. This week's graph is **unweighted**: its edges carry no distance, duration or

other numerical cost. The path length in this graph is a count of edges, not metres.

Use this small graph for the worked traces in Sections 2–6. S–B–G uses two edges, whereas S–A–C–D–G uses four. Both are valid paths. Each edge counts as one allowed transition, regardless of its drawn length. The Assignment applies the same functions to a larger supplied graph with branches and additional cycles.

A graph is **connected** when every pair of nodes can be joined by a path in the undirected graph. The six nodes S, A, B, C, D and G in the running graph form a connected component, but the whole graph is not connected because X is isolated. A **cycle** returns to its starting node without repeating an edge or any other node. The connection S–A–C–D–G–B–S is a cycle. Going S–A–S merely retraces the same undirected edge and is not a cycle in this example. Remembering visited nodes prevents exploration from continuing indefinitely around cycles or back along connections already explored.

An **undirected edge** permits movement in both directions. In this graph, the **degree** of a node is the number of edges incident to it: S has degree two, and X has degree zero. A **directed edge** permits only the direction shown by its arrow. For example, S → A and S → B give S an out-degree of two. If those are the only arrows, S has in-degree zero, and A has in-degree one and out-degree zero. Out-degree counts arrows leaving a node. In-degree counts arrows entering it. The direction of a connection must come from the input, not from where a node happens to appear on the page.

1.1 A tree gives each non-root node one parent

A **tree** is an undirected connected graph with no cycle. Choose a **root** to describe a hierarchy. In the right-hand tree in Figure 1, S is the root. The **parent** of a node is its immediate predecessor on the unique path from the root. Thus A and B have parent S, C has parent A, D has parent C, and G has parent B. The root has no parent.

A node's **children** are the nodes for which it is the parent. S has children A and B. A **leaf** has no children, so D and G are leaves in this rooted tree. The **depth** of a node is the number of tree edges from the root. S has depth zero, C has depth two, and D has depth three. A **subtree** contains a chosen node and all its descendants. The subtree rooted at A contains A, C and D.

The original graph may have several paths to a node. A **traversal** systematically visits and processes nodes reachable through the supplied connections. It records only the predecessor chosen when that node is first accepted. These parent links form a **traversal tree** rooted at the start. A **search tree** records reached states and the selected predecessor edges, so the BFS and DFS traversal trees are the search trees built by those algorithms. They do not remove edges from the input graph. Different traversal rules can produce different trees from the same graph. X has no parent in a traversal started at S because that traversal never reaches X.

2 Representing connections in Python

The supplied file `src/ap_week09/cases.py` contains `GRAPH`. Each dictionary key is one node ID. Its value is an ordered tuple of neighboring node IDs. This is an **adjacency-list representation** implemented with a dictionary of tuples: each node stores its neighbors. The name describes the organization of connections and does not require every neighbor collection to be a Python `list`. These values are neighbors, not a path that the robot should immediately follow.

Code — supplied graph in src/ap_week09/cases.py

```
GRAPH = {
    "S": ("A", "B"),
    "A": ("S", "C"),
    "B": ("S", "G"),
    "C": ("A", "D"),
    "D": ("C", "G"),
    "G": ("B", "D"),
    "X": (),
}
```

For example, `GRAPH["S"]` returns `("A", "B")`. The declared neighbor order is A, then B. The tuple does not require A to be followed immediately by B in the final path. The empty tuple at X means X is a valid node with no outgoing connections. It does not mean that X is missing from the graph.

Each undirected edge appears in two tuples. A occurs among S's neighbors, and S occurs among A's neighbors. There are seven dictionary keys and twelve neighbor entries, representing six undirected edges. A directed graph need not contain the reverse entry. The traversal algorithms therefore follow exactly the entries given rather than adding reverse connections themselves.

2.1 Read supplied input before implementing a search

After preparing the repository as described in the Assignment, run `python examples/inspect_graph.py` from its root directory. This complete supplied program validates and reads `GRAPH` and `PARENT_EXAMPLE`. It does not call any unfinished algorithm. It can therefore run before the three TODO bodies are completed.

Expected output — supplied inspection

```
neighbors of S: ('A', 'B')
neighbors of X: ()
node count: 7
undirected edge count: 6
supplied parent mapping: {'S': None, 'B': 'S', 'G': 'B'}
parent of G: B
parent of B: S
parent of S: None
```

The expression `len(GRAPH)` returns the node count. Reading `GRAPH["X"]` returns an empty tuple. Reading a parent dictionary returns one predecessor, rather than all neighbors. These are ordinary dictionary operations already used with named records. The difference is the meaning of the stored value.

The supplied checks reject an empty graph, a missing start, repeated neighbors within one tuple, or a neighbor ID without its own dictionary key. A supplied workflow also checks that the requested goal is a graph node. These checks are complete support. Student traversal code receives the validated graph and does not have to implement a new validation framework.

3 Breadth-first search

Breadth-first search (BFS) explores nodes in increasing numbers of edges from the start. Its **frontier** contains discovered nodes waiting to be processed. Use a deque as a FIFO queue: append on the right and remove with `popleft()` on the left. Earlier discoveries are processed first.

Mark a node `visited` and record its parent when first adding it to the queue. This prevents duplicate additions and

preserves its first parent. Initialize the queue with S, mark S visited and set its parent to `None`. On removal, append the node to `order` and inspect its neighbors in tuple order.

Figure 2 shows the first three expansions. Read each queue from left to right. A gray node is already visited even if it is still waiting in the queue.

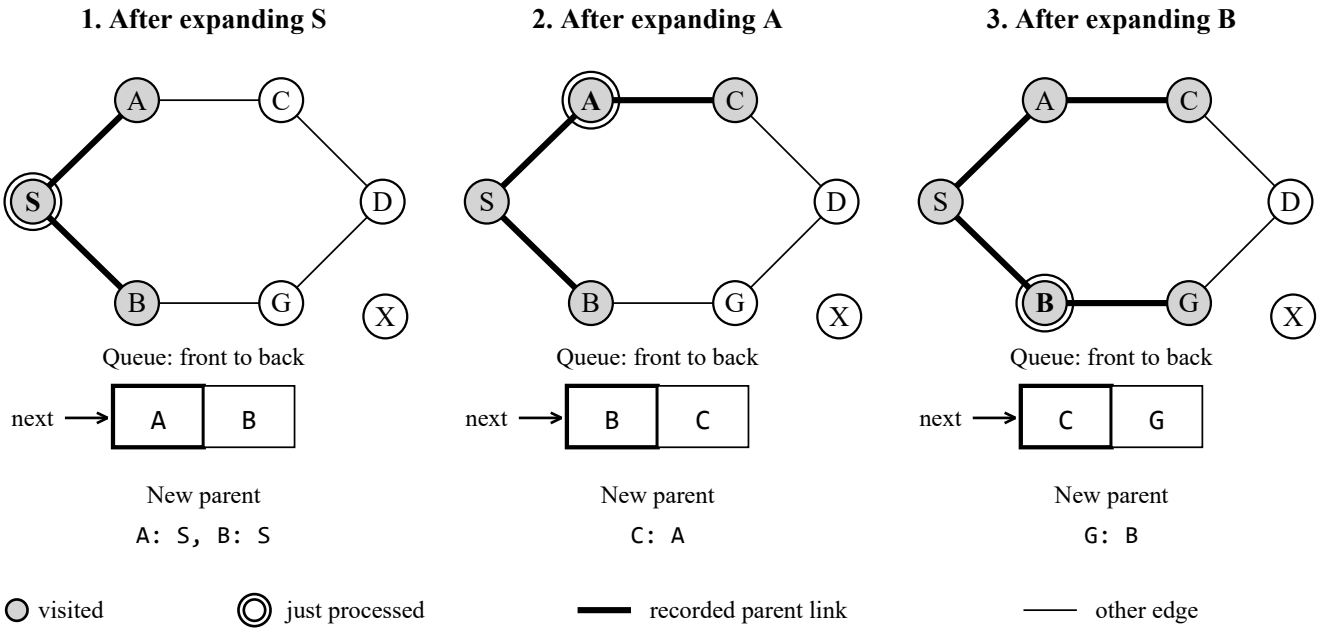


Figure 2: BFS spreads outward. After S adds A and B, processing A adds C behind B. B is therefore processed before C and discovers G. Each panel shows the state after removal and neighbor additions.

Table 1 completes the same trace. Start shows initialization. Each later row follows one expansion. "New parent" lists only that step's additions.

Table 1: BFS frontier and first-parent changes from S.

Processed	Queue: front → back	Newly visited	New parent
Start	[S]	S	S: None
S	[A, B]	A, B	A: S, B: S
A	[B, C]	C	C: A
B	[C, G]	G	G: B
C	[G, D]	D	D: C
G	[D]	none	none
D	[]	none	none

Continue until the queue is empty, giving order S, A, B, C, G, D. After C discovers D, G cannot replace D's first parent. The G–D edge still exists but is not a parent link.

BFS processes one-edge discoveries before two-edge discoveries, so G's first parent chain gives the minimum-edge path S–B–G. This does not guarantee the shortest physical distance or time: those costs are absent from the graph.

4 Depth-first search

Depth-first search (DFS) follows one branch deeply before returning to alternatives. Use the deque as a LIFO stack: append and `pop()` both operate on the right. The last added candidate is removed next. This implementation does not require recursion.

Each stack entry is `(node, parent_candidate)`: a node and its proposed predecessor. Start with `("S", None)`. Unlike BFS, mark visited only on the first accepted pop, then record the parent and append the node to `order`. Skip a later entry for an already visited node.

Push unvisited neighbors in reverse tuple order using `reversed(neighbors)`, without changing the tuple. For S's (A, B), push B first, then A. A is now on top and is explored first, while B waits below it.

Figure 3 shows how the next candidate stays on the current branch. A pending pair is not a visit or a recorded parent link.

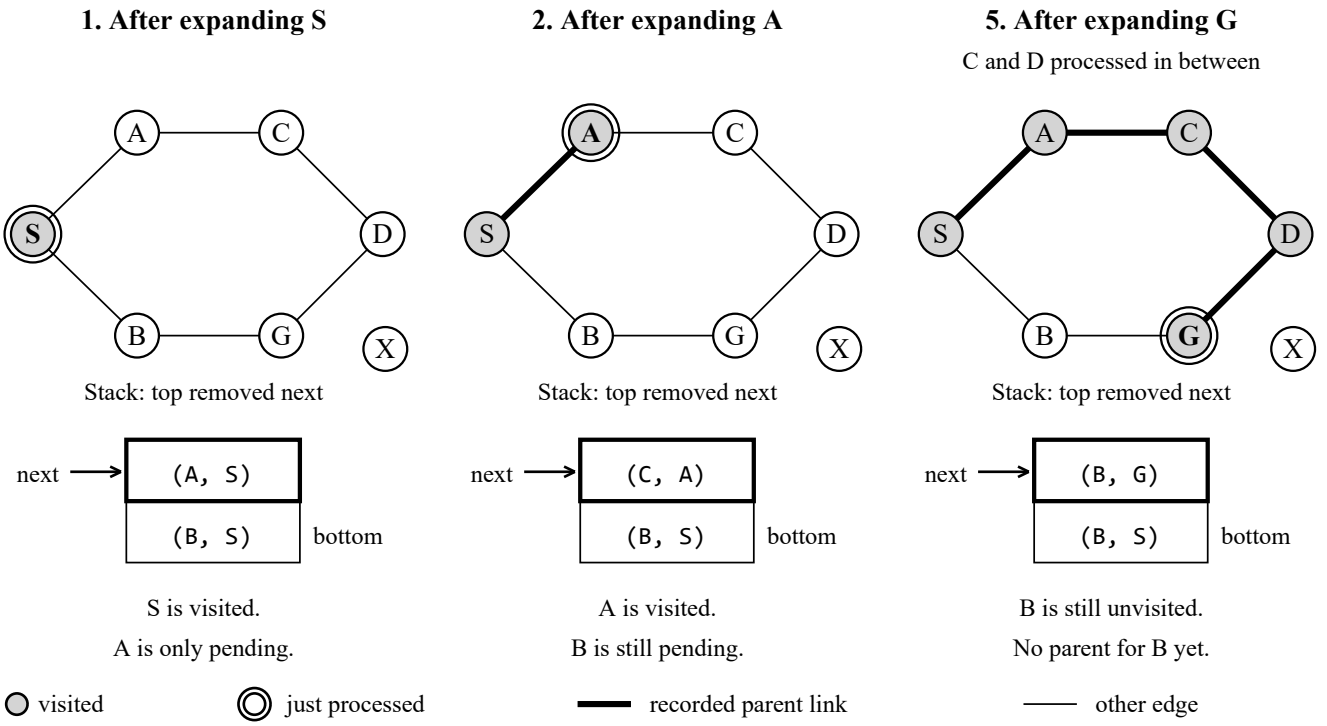


Figure 3: DFS follows S–A–C–D–G while the early B candidate waits. After G is expanded, (B, G) is on top of (B, S). B is still unvisited. The next pop records parent[B] = G. Panels show expansions 1, 2 and 5, not consecutive steps.

Table 2 supplies every step. Stack entries run from bottom to top, so the rightmost pair is removed next. Start shows initialization. Later rows include the pop and any resulting pushes.

Table 2: DFS candidate stack and finalized parent changes from S.

Popped pair	Stack: bottom → top	Finalized parent
Start	[(S, None)]	none
(S, None)	[(B, S), (A, S)]	S: None
(A, S)	[(B, S), (C, A)]	A: S
(C, A)	[(B, S), (D, C)]	C: A
(D, C)	[(B, S), (G, D)]	D: C
(G, D)	[(B, S), (B, G)]	G: D
(B, G)	[(B, S)]	B: G
(B, S)	[]	none: skip B

The order is S, A, C, D, G, B. After (B, G) fixes B's parent, the older (B, S) is skipped. Assigning B's parent on its earlier push would violate this first-pop contract.

DFS reaches G in four edges here, compared with BFS's two. DFS does not promise minimum-edge paths. Both traversals expand each reachable node once, preserve the graph and leave the unreachable X unvisited.

4.1 Traversal order depends on the stated rule

Reverse only S's neighbor tuple to (B, A), keeping every other tuple fixed. BFS then processes S, B, A, G, C, D. Its parent of D changes to G, but its path from S to G still has two edges. DFS processes S, B, G, D, C, A and now also reaches G in two edges. A short path in this one DFS run does not establish a general shortest-path guarantee.

For an adjacency-list representation, each reachable node is expanded once and its neighbor entries are inspected once. With average constant-time set and dictionary operations, the traversal core takes $O(|V| + |E|)$ time. An undirected edge supplies two neighbor entries, which changes the constant factor, not this order of growth. The running graph has $|V| = 7$ and $|E| = 6$, although a traversal from S reaches only six nodes. BFS uses $O(|V|)$ auxiliary storage. This candidate-pair DFS can retain duplicate pending entries, so its stack can require $O(|E|)$ space in addition to $O(|V|)$ visited/order/parent storage. These bounds describe the traversal core, not all supplied input validation, drawing or simulation work.

5 Reconstructing a path from parents

The traversal order is not the requested start-to-goal path. BFS processes A and C before G, but its path to G goes through B. A `parent` dictionary records the predecessor of each reached node. Reading `parent["G"]` gives `"B"`, not a complete path. Repeating that lookup lets us recover the path.

`reconstruct_path(parent, start, goal)` receives this dictionary as input and returns a new list of node IDs. It does not build the dictionary or run BFS or DFS. The supplied `PARENT_EXAMPLE` in `cases.py` makes this operation testable before either traversal is implemented. This short illustration shows the call and its result after TODO 1 is complete:

Code — reconstruction call using supplied values

```
parent = {"S": None, "B": "S", "G": "B"}
path = reconstruct_path(parent, "S", "G")
print(path)
```

Expected output

```
['S', 'B', 'G']
```

Start with G as the current node. At each step, append the current node to a new list, then read its parent to obtain the next node. The table separates the node being saved from the value returned by the lookup.

Current node	List after append	Next node from lookup
`"G"`	`["G"]`	`parent["G"]` returns `"B"`
`"B"`	`["G", "B"]`	`parent["B"]` returns `"S"`
`"S"`	`["G", "B", "S"]`	`parent["S"]` returns `None`

`None` ends the chain and is not appended. Reverse [G, B, S] to return [S, B, G]. The caller assigns this returned list to `path`, and `print(path)` displays it. The input dictionary stays unchanged. When start equals goal, return [start]. If goal is absent from the valid parent mapping, return `None`.

X is a known but unreachable graph node, so reconstruction returns `None`. The full workflow rejects a goal outside the graph. Supplied parent validation rejects malformed chains. Physical execution is a separate check. Following and reversing a k-node chain takes $O(k)$ time and $O(k)$ output storage, excluding supplied validation.

6 A graph path becomes a robot motion

Each node now names a stored robot-arm posture. Figure 4 connects the BFS result [S, B, G] to the corresponding postures. The search chooses their order. The supplied Genesis program moves the arm through them.

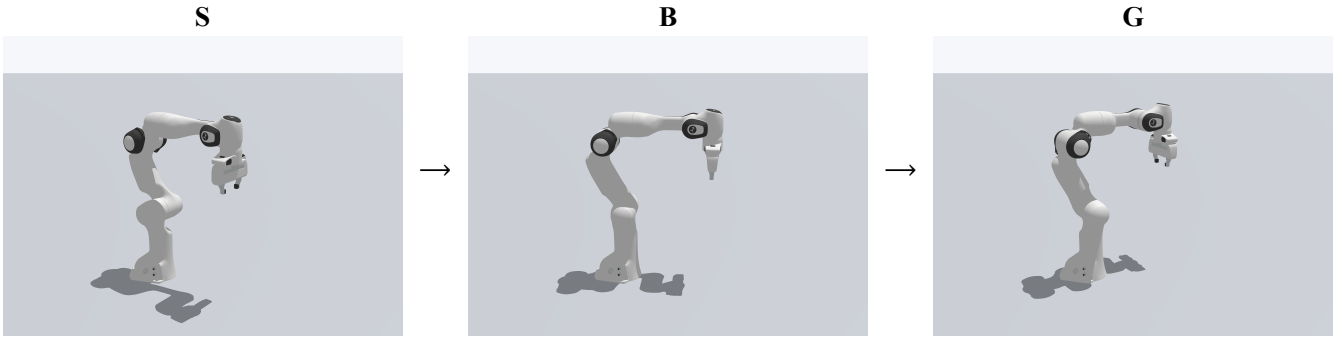


Figure 4: One node, one stored posture. Actual Genesis frames show the arm at S, B and G. The arrows follow the two graph edges of the BFS path, not measured straight-line motion of the robot hand.

The supplied `POSTURES` dictionary in `src/ap_week09/robotposes.py` maps each ID to seven joint angles in radians. For example, `POSTURES["B"]` returns the targets for posture B. These are joint angles, not the metre-valued (x, y) coordinates used in earlier weeks. Students do not calculate these targets or implement robot control.

The runner `examples/run_robot_path.py` calls `solve(graph, "S", "G", args.algorithm)`, where `args.algorithm` is the selected `bfs` or `dfs` option. It receives `order, parent, path`. This supplied function connects the student's traversal and reconstruction. The robot runs use the larger Assignment graph introduced next. Its BFS path is still [S, B, G], but its DFS path includes additional postures. The runner looks up each posture in the recovered path, not every visited node. If `path` is `None`, it reports `NO_PATH` without starting motion.

Only the declared graph edges are available to the search. Do not add a direct S–G connection because the pictured postures seem close. BFS minimizes the number of these edges, not physical distance or movement time. The supplied simulation checks tracking and observed contacts, but does not establish safety for arbitrary robot motions.

GPU execution is required. Reuse the existing `applied-programming-cuda` environment from the Week 5 CUDA/Genesis guide. The Assignment gives the two required commands after the algorithms are complete. Keep the separate `applied-programming-w06` environment for graph checks. Do not create a new environment or replace the required GPU runs with CPU execution.

7 Assignment: find and execute a graph path

7.1 Purpose and work to complete

Implement path reconstruction, BFS and iterative DFS. Apply the same three functions to the supplied 14-node graph `ASSIGNMENT_GRAPH` in `src/ap_week09/cases.py`. Thirteen nodes are reachable from S, while X is isolated. H, J, K and L are dead ends. The loops provide alternative ways to reach a node, so visited-node tracking and parent selection matter. The recovered path, not the full visitation order, drives the supplied robot arm.

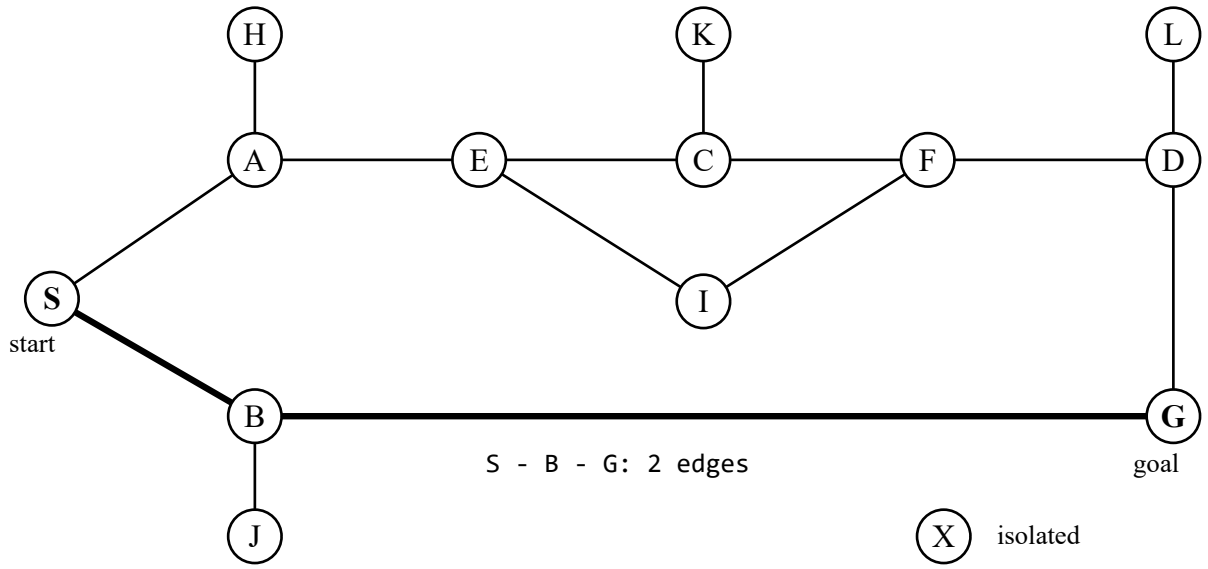


Figure 5: Assignment graph: 14 nodes and 14 undirected edges. The bold route S–B–G uses two edges. DFS with the supplied neighbor order reaches G along S–A–E–C–F–D–G, using six edges. Dead-end nodes can be visited without appearing in either recovered goal path. The drawing shows connections, not physical distances.

The three stage commands below retain the small worked example so each implementation can be checked separately. The final checks, comparison command and required robot runs use the larger Assignment graph. No additional student function is required.

Student file — src/ap_week09/traversal.py

Replace `raise NotImplementedError(...)` inside the three marked regions. Keep the function headers, imports, validation and BEGIN/END markers. Graph data, tests and robot execution are provided. No test file, report or screenshot submission is required.

7.2 Prepare the repository

Open the Week 9 starter repository:

https://github.com/SSUMechE/applied-programming-2026-week09-starter

Use `Use this template` to create a private `applied-programming-w09-YOUR-STUDENT-ID` repository and invite `SSUMechE`. Replace the folder, account and student ID below with your own values.

Run — Anaconda Prompt

```
cd /d "C:\your-course-folder"
git clone https://github.com/YOUR-GITHUB-ID/applied-programming-w09-YOUR-STUDENT-ID.git
cd applied-programming-w09-YOUR-STUDENT-ID
git remote -v
set PYTHONUTF8=1
conda activate applied-programming-w06
python -m pip install -e . --no-build-isolation
python scripts/verify_environment.py
python examples/inspect_graph.py
```

Both origin addresses must name your repository. Expect `ENVIRONMENT PASS` and the Section 2.1 graph output before completing the algorithms. Keep the existing package versions. An import error is a setup problem.

7.3 TODO 1: reconstruct a path

Edit — traversal.py, reconstruct_path, TODO 1

Use the `parent` dictionary already passed to `reconstruct_path(parent, start, goal)`. Each key is a reached node, and its value is that node's predecessor. The existing entry for `start` has value `None`. The strings `start` and `goal` name the requested endpoints. Create and return a new path list, without constructing another parent mapping or running a traversal here.

1. If `goal` is not a key in `parent`, return `None`.
2. Otherwise, begin at `goal`. Collect each current node in a new list and follow its parent until `None` is reached. Do not put `None` in the list.
3. Reverse the collected nodes and return the list in start-to-goal order. When start equals goal, the returned list contains only start.

Keep `parent` unchanged. Return the result rather than printing it. Input validation is supplied. Section 5 traces the same dictionary lookups. The following command calls only this function with the supplied mapping, so TODOs 2 and 3 may remain unfinished.

Run — repository root, applied-programming-w06

```
python examples/compare_traversals.py --stage reconstruct
```

Expected output

```
S -> G: ['S', 'B', 'G']
S -> S: ['S']
S -> X: None
```

7.4 TODO 2: breadth-first search

Edit — traversal.py, bfs, TODO 2

In `bfs(graph, start)`, `graph` and the start-node ID are supplied inputs. Create `order` (a processed-node list), `visited` (a set), `parent` (a predecessor dictionary) and a deque queue. Here `parent` is an output you build, unlike the input dictionary in TODO 1.

1. Start with an empty `order`, a queue containing `start`, `start` in `visited`, and its parent set to `None`.
2. Remove nodes with `popleft()`. Append each removed node to `order` and inspect its neighbors in tuple order.
3. Add each unvisited neighbor at the queue's right end. At that moment, mark it visited and record the current node as its first parent.
4. Continue until the queue is empty and return `(order, parent)`.

Preserve `graph` and do not print. There is no goal argument or early stop at G. Use the queue trace in Section 3. This stage calls only BFS and prints its returned list and dictionary.

Run — repository root, applied-programming-w06

```
python examples/compare_traversals.py --stage bfs
```

Expected output

```
order: ['S', 'A', 'B', 'C', 'G', 'D']
parent: {'S': None, 'A': 'S', 'B': 'S', 'C': 'A', 'G': 'B', 'D': 'C'}
```

7.5 TODO 3: depth-first search

Edit — traversal.py, dfs, TODO 3

In `dfs(graph, start)`, `graph` and the start-node ID are supplied inputs. Create a new `order` list, `visited` set, `parent` dictionary and deque stack. Return `(order, parent)` after exploring every reachable node. Each stack entry pairs a node with its proposed predecessor.

1. Leave `order`, `visited` and `parent` empty initially. Put only `(start, None)` in the stack.
2. Remove the rightmost pair with `pop()`. If its node is already visited, skip it. Otherwise, mark the node visited, record the pair's predecessor as its parent, and append the node to `order`.
3. Push pairs for unvisited neighbors in reverse tuple order. Each pair contains the neighbor and the current node. A waiting pair is not a visit, so several pairs may name the same node.
4. Continue until the stack is empty, then return `(order, parent)`.

Preserve `graph` and do not print. Use the stack trace in Section 4: B's parent becomes G when (B, G) is popped, not S when (B, S) is first pushed. This stage calls only DFS.

Run — repository root, applied-programming-w06

```
python examples/compare_traversals.py --stage dfs
```

Expected output

```
order: ['S', 'A', 'C', 'D', 'G', 'B']
parent: {'S': None, 'A': 'S', 'C': 'A', 'D': 'C', 'G': 'D', 'B': 'G'}
```

7.6 Check the algorithms and execute both paths

After all three functions are complete, run the supplied nine-case suite, graph comparison and file check. Implement the stated rules for any validated input, not just the example IDs.

Run — repository root, applied-programming-w06

```
python -m pytest -q
python examples/compare_traversals.py
python scripts/verify_submission.py
```

Expect `9 passed` and `LOCAL FILE CHECK PASS`. Unlike the stage commands, this comparison uses the larger Assignment graph. It holds S and G fixed and shows these paths:

- Original Assignment graph: BFS [S, B, G], DFS [S, A, E, C, F, D, G].
- Reverse only S's neighbor order: both return [S, B, G].
- Remove B–G: both return [S, A, E, C, F, D, G].
- Also remove D–G: both return `None`.

Compare the visitation order with the recovered path. H, J, K and L are visited from S but do not belong in its path to G. Closing B–G removes the shortcut. Closing D–G as well makes G unreachable. If a check fails, correct the relevant function and rerun that check. No copied output record is required.

GPU execution is required for the two original paths. Activate the existing CUDA/Genesis environment in the same repository. The install command exposes this package without replacing its dependencies.

Run — repository root, applied-programming-cuda

```
conda activate applied-programming-cuda
python -m pip install -e . --no-deps --no-build-isolation
python examples/run_robot_path.py --algorithm bfs --output artifacts/bfs --capture
python examples/run_robot_path.py --algorithm dfs --output artifacts/dfs --capture
```

Both runs must report `Result: EXECUTED`. Compare the reached-node images in `artifacts/bfs` and `artifacts/dfs`: the former follows S–B–G, the latter S–A–E–C–F–D–G. The runner supplies joint tracking, checks and recording. Generated files are for inspection and are not submission items. A CUDA or simulator error is distinct from a wrong node sequence. Do not replace the required GPU run with CPU execution.

If further algorithm checks are needed, return to the existing core environment:

```
conda activate applied-programming-w06
```

7.7 Push and submit

Review and push your completed algorithm file. Keep supplied files unchanged and do not commit environments, caches or credentials.

Run — repository root

```
git status --short
git diff
git add src/ap_week09/traversal.py
git diff --staged
git commit -m "Complete Week 9 graph traversal"
git push origin main
git status --short
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

Press `q` to leave a diff pager. The final status must be empty, and the full local and remote commit IDs must match. Inspect the pushed revision on GitHub.

- Keep the required Week 9 repository name and Private visibility. Invite `SSUMechE`. A correct invitation sent on time but awaiting instructor acceptance is not a student omission.
- Within one week of the lab, by the exact LMS date and time, submit your repository URL and the pushed revision's full 40-character commit ID.
- GitHub upload alone is not LMS submission. Later pushes do not replace the submitted commit. No extra LMS confirmation sentence, ZIP, report or screenshot is required.

Submission example — replace both fictional values

```
Repository URL: https://github.com/student-example/applied-programming-w09-20260000
Commit ID: 0123456789abcdef0123456789abcdef01234567
```

The announced zero-point rule applies to missing mandatory submission requirements at the deadline, with the timely-pending invitation exception above. A failed algorithm test alone is not automatic zero for the entire Assignment.